

CC002.23: Final Project: Movie Ticket Booking System



In Fulfillment of the Requirements for
The CC002.23: Computer Programming 1
Bachelor of Science in Computer Science

Proponents:

Art Xenos D. Guadalquiver

Althea Denielle M. Javillo

Course and Section:

BS Computer Science — 1A

Submitted to:

Mr. Michel Bartolome Bolo

Problem Description / Background

Customer service in the contemporary digital age relies on convenience and speed. Similar to booking for hotels, plane tickets, seat and booking management remains an issue for cinemas, especially during peak times or top shows. The customer experience is disrupted by manual bookings and long waiting lines, and can lead to double bookings and delays.

By applying a straightforward console-based Java application to book the tickets automatically, the Movie Ticket Booking System aims to sort out such issues. Audience members can select their preferred seat, choose a time, and display a list of the available films. The system calculates the cost and reserves a seat as unavailable.

The system effectively handles a large number of seats and screenings and keeps track of seat availability using arrays and nested for loops. Information can be stored even when the system has been closed due to file handling, which ensures that reservation records of seats are stored and retrieved every time the application is run. In addition, the system utilizes methods to dissect certain features.

Objectives of the Program

The main objective of this project is to develop a functional, console-based Movie Ticket Booking System that applies key Java programming concepts to simulate a real-world reservation process. Specifically, it aims to:

1. Allow users to select movies, showtimes, and seats at their own discretion through self-service.
2. Apply key Java programming principles, specifically arrays assess movie listings, seat availability, and user input.
3. Implement nested loops for displaying and managing the two-dimensional seat layout and validating seat selections.
4. Incorporate file handling to store and retrieve seat reservation data, ensuring that previously booked seats remain marked as reserved even after the program restarts.

Features and Functions:

1. Selection of Movie Titles and Time.
Allows users to select available movie screenings and time schedules.
2. Multibookings
Users can select a minimum of 1 person to a maximum of 5 persons in one booking. Users can also repeat the booking process or exit booking.
3. Seat Reservation
Users are offered a visual display of available and occupied seats for specific movie screenings. The program also has evaluation for open spaces specific to the movie and time.
4. Manage Bookings
Total cost and seat availability are saved using file handling. This data can be retrieved when the program restarts.
5. Booking Summary
The user's input is organized in a display after completing a booking. It includes selected movie, time, seat row, and total cost.

Program Flow: Pseudocode

START

totalCost = 0

rebooking = true

totalCost = readTotalCostFromFile("total_cost.txt")

WHILE rebooking DO

 PRINT welcome message

 DISPLAY movies (1-3)

 GET movieSelection (1-3)

 DISPLAY times (1-4)

 GET timeSelection (1-4)

 seatFile = "seats_" + movieSelection + "_" + timeSelection + ".txt"

 seats = initializeSeatsArray(4x5, 'O')

 seats = loadSeatsFromFile(seatFile, defaultSeats)

GET numberOfPeople (1-5)

availableSeats = countAvailableSeats(seats)

IF availableSeats < numberOfPeople THEN

 PRINT "Not enough seats"

 CONTINUE

END IF

DISPLAY seat layout

FOR each person FROM 1 TO numberOfPeople DO

 booked = false

 WHILE NOT booked DO

 GET row (1-4)

 GET seat (1-5)

 IF seat occupied THEN

 PRINT "Seat taken"

 ELSE

 MARK seat 'X'

 RECORD seat details

 PRINT "Booked"

 booked = true

 DISPLAY updated layout

 END IF

 END WHILE

END FOR

bookingCost = numberOfPeople * 350

totalCost = totalCost + bookingCost

saveSeatsToFile(seatFile, seats)

saveTotalCostToFile("total_cost.txt", totalCost)

DISPLAY summary (movie, time, seats, costs)

GET rebookChoice (1=Yes, 2=No)

IF rebookChoice == 2 THEN

 rebooking = false

 PRINT "Thank you"

END IF

END WHILE

CLOSE input

FUNCTION countAvailableSeats(seats)

available = 0

FOR each row IN seats DO

FOR each seat IN row DO

IF seat == 'O' THEN

available++

END IF

END FOR

END FOR

RETURN available

END FUNCTION

FUNCTION displaySeatLayout(seats)

PRINT "Layout:"

FOR rowIndex FROM 0 TO 3 DO

PRINT "Row " + (rowIndex+1) + ": "

FOR each seat IN row DO

PRINT seat + " "

END FOR

PRINT newline

END FOR

IF available == 0 THEN

PRINT "All booked"

END IF

END FUNCTION

FUNCTION getValidatedInput(message, limit)

WHILE true DO

PRINT message

TRY

choice = GET int

IF 1 <= choice <= limit THEN

RETURN choice

ELSE

PRINT "Invalid"

END IF

CATCH

PRINT "Invalid"

CONSUME input

END TRY

```
END WHILE
END FUNCTION
```

```
FUNCTION loadSeatsFromFile(filePath, defaultSeats)
    TRY
        OPEN file
        loadedSeats = new 4x5 array
        row = 0
        WHILE hasLine AND row < 4 DO
            line = READ line
            seatData = SPLIT by ","
            IF length != 5 THEN
                RETURN defaultSeats
            END IF
            FOR col FROM 0 TO 4 DO
                loadedSeats[row][col] = seatData[col][0]
            END FOR
            row++
        END WHILE
        RETURN loadedSeats
    CATCH
        RETURN defaultSeats
    END TRY
END FUNCTION
```

```
FUNCTION saveSeatsToFile(filePath, seats)
    TRY
        OPEN file
        FOR each row IN seats DO
            FOR col FROM 0 TO 4 DO
                WRITE seats[row][col]
                IF col < 4 THEN
                    WRITE ","
                END IF
            END FOR
            WRITE newline
        END FOR
    CATCH
        PRINT "Save error"
    END TRY
END FUNCTION
```

```
FUNCTION readTotalCostFromFile(filePath)
```

```
    TRY
```

```
        OPEN file
```

```
        IF hasInt THEN
```

```
            RETURN int
```

```
        ELSE
```

```
            RETURN 0
```

```
        END IF
```

```
    CATCH
```

```
        RETURN 0
```

```
    END TRY
```

```
END FUNCTION
```

```
FUNCTION saveTotalCostToFile(filePath, totalCost)
```

```
    TRY
```

```
        OPEN file
```

```
        WRITE totalCost
```

```
    CATCH
```

```
        PRINT "Save error"
```

```
    END TRY
```

```
END FUNCTION
```

```
FUNCTION displayBookingSummary(movie, time, bookedSeats, people, bookingCost, totalCost)
```

```
    seatDetails = ""
```

```
    FOR i FROM 0 TO people-1 DO
```

```
        IF i > 0 THEN
```

```
            seatDetails += newline
```

```
        END IF
```

```
        seatDetails += "Person " + (i+1) + " " + bookedSeats[i]
```

```
    END FOR
```

```
    PRINT summary with movie, time, seatDetails, bookingCost, totalCost
```

```
END
```

Screenshots of Program Output:

- **Menu Display and User Input/Selection**

```
=====
MOVIE BOOKING SYSTEM
=====

Welcome to Hall of Cinema!
Now Showing:

1. Avatar: Fire and Ash
2. Barbie
3. Oppenheimer
```

```
Select movie title (1-3): 1

1. 12:00 PM
2. 3:00 PM
3. 6:00 PM
4. 9:00 PM

Select time (1-4): 2
No previous seat data found. Load default.
Input number of people in the booking (1-5): 2
```

- **If Invalid Inputs**

```
Select movie title (1-3): 6
Invalid input. Please choose between 1-3

Select movie title (1-3): a
Invalid input. Please enter a number.
```


- **Seat Availability and Selection**

```
Seat Layout (O = Open | X = Occupied):
```

```
Row 1: O O O O O
```

```
Row 2: O O O O O
```

```
Row 3: O O O O O
```

```
Row 4: O O O O O
```

```
Booking seat for Person 1:
```

```
Enter row number (1-4): 1
```

```
Enter seat number (1-5): 1
```

```
Seat booked successfully for Person 1!
```

```
Seat Layout (O = Open | X = Occupied):
```

```
Row 1: X O O O O
```

```
Row 2: O O O O O
```

```
Row 3: O O O O O
```

```
Row 4: O O O O O
```

- **If Insufficient/Full**

```
Input number of people in the booking (1-5): 5
```

```
Not enough seats available for 5 people. Only 2 seats available.
```

```
Booking cannot proceed for all persons.
```

```
=====
```

```
MOVIE BOOKING SYSTEM
```

```
=====
```

```
Welcome to Hall of Cinema!
```

```
Now Showing:
```

```
1. Avatar: Fire and Ash
```

```
2. Barbie
```

```
3. Oppenheimer
```

- **Booking Summary and Rebooking Option**

- **If Yes:**

```
Do you want to book again? (1 = Yes | 2 = No): 1
=====
                MOVIE BOOKING SYSTEM
=====

Welcome to Hall of Cinema!
        Now Showing:

1. Avatar: Fire and Ash
2. Barbie
3. Oppenheimer
```

- **If No:**

```
===== BOOKING SUMMARY =====
Movie: Avatar: Fire and Ash
Time : 3:00 PM
Person #1 Row 1, Seat 1
Person #2 Row 1, Seat 2
Booking Cost: 700 PHP
=====
Do you want to book again? (1 = Yes | 2 = No): 2

Thank you for booking with us!
```

- **If Invalid**

```
Do you want to book again? (1 = Yes | 2 = No): 3
Invalid input. Please select 1 for Yes or 2 for No.
Do you want to book again? (1 = Yes | 2 = No): ||
```

Conclusion and Learnings:

The Movie Ticket Booking System is fundamentally focused on seat data and user inputs, and developing it enabled us to understand how various programming concepts integrate to form an effective and responsive application. At first, organizing the logic and connecting components proved challenging, particularly when incorporating arrays, nested loops, and file handling. As development progressed, we learned to structure the program flow more effectively by using methods to organize distinct tasks such as displaying movies, selecting seats, calculating ticket costs, and saving booking records.

This project consisted mostly of if-else statements to validate user input and minimize runtime errors. For example, when a user selects an invalid or already booked seat, the program displays an error message and requests new input. Arrays were essential for storing seat information, while nested loops enabled efficient display and updating of the seating chart in the console. File handling in the system allows better saving and retrieving seat availability and booking data. The program logic, which included movie selection, showtime choice, seat validation, and recalculating total cost, heavily implies user input in order to process. The use of methods further enhanced modularity, organization, and reusability.

Debugging and testing in each error improved our understanding of the program's control flow and component interactions. For example, the feature of seat availability with the integration of file handling, as well as the implementation of multiple bookings, was challenging to resolve, and there were multiple tests in order the seat data in each movie and time is unique. Overall, this improved our proficiency in programming concepts particularly loops, conditionals, arrays, file handling, and methods into a practical application.

The program simulates a cinema ticket booking process in which users select a movie, showtime, and seats. The system tracks seat availability for each show to prevent overbooking and persistently accumulates total costs. It uses file input and output to maintain data between sessions, robustly validates user input, and provides real-time seat updates in a loop until the user chooses to exit.

The code, written in Java, uses modular static methods for input validation, seat display, file operations, and summary generation. Arrays represent movies, showtimes, and seating charts; loops manage repeated user interactions and seat display, and conditions ensure reliable bookings. Input and output rely on console-driven prompts that allow us to experiment through its basic text interface.

For future improvements, the system could expand dynamically with databases, introduce GUI or web-based interfaces, add payment and user authentication features, improve error management, and optimize performance for concurrency. We must apply deep scrutiny, especially in bookings and menus, increasing accessibility beyond manual transactions, aiding business decision-making, and serving as a foundational prototype for more advanced theater booking platforms.